Buffer Cache and I/O Scheduling

Efficient I/O Management in Operating Systems

Prashant Jadav

DEPSTAR - CHARUSAT

Why Is I/O Management Important?

- CPU and main memory are generally much faster than storage and many I/O devices.
- Poor I/O management can leave the CPU waiting for slow devices.
- Operating systems therefore use:
  - **Buffering** to handle speed differences and temporary data storage.
  - **Caching** to avoid repeated access to slow devices.
  - **I/O scheduling** to determine the order in which requests are serviced.

Goal

Improve **throughput**, reduce **latency**, minimize unnecessary device movement, and use memory and I/O resources efficiently.

Example

A disk may serve many processes simultaneously. The OS can reorder pending requests to reduce disk-head movement and improve overall response time.

Buffering, Caching and Spooling

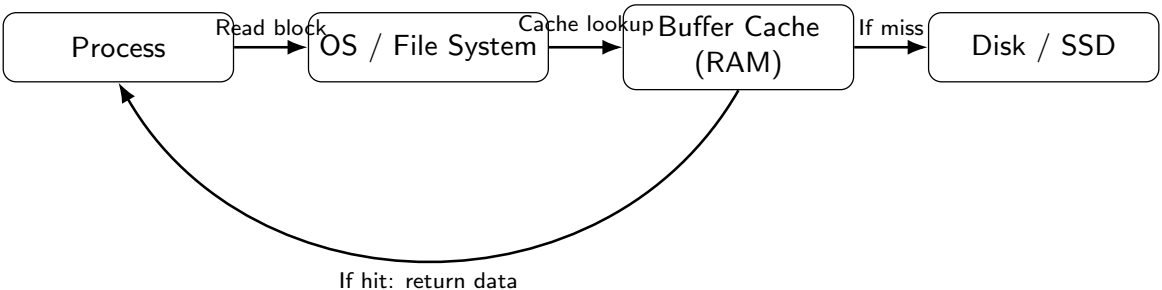
Technique	Purpose	Example
Buffering	Temporarily hold data while producer and consumer operate at different rates	Network data accumulated in a receive buffer
Caching	Keep frequently/recently used data in faster storage	Disk blocks kept in RAM
Spooling	Queue work for a device that serves requests sequentially	Print jobs stored in a print queue

Important distinction

A **buffer** primarily smooths a rate mismatch; a **cache** aims to exploit locality and avoid repeated expensive accesses.

What Is a Buffer Cache?

- A **buffer cache** is a region of main memory used to hold recently accessed disk blocks.
- When a process requests a block, the OS first checks whether the block is already in memory.
- If present, the request can be satisfied without accessing the disk.
- If absent, the OS reads the block from storage and places it in the cache.



Buffer Cache Hit and Miss

Cache Hit

1. Process requests block B.
2. OS searches the cache.
3. Block B is found.
4. Data is returned from RAM.

Cache Miss

1. Process requests block B.
2. OS searches the cache.
3. Block B is absent.
4. Storage device is accessed.
5. Block is loaded into cache.

Performance intuition

If the cache hit ratio is high, many requests avoid slow storage access.

$$T_{\text{avg}} \approx hT_{\text{cache}} + (1 - h)T_{\text{storage}}$$

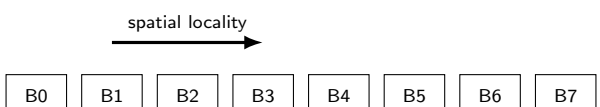
where h is the cache hit ratio.

Why Does Buffer Cache Work? Locality

- Programs tend to access data with **locality**.
- **Temporal locality:** recently accessed data may be accessed again soon.
- **Spatial locality:** data near a recently accessed location may be accessed soon.

File-system example

A program reading a file sequentially accesses consecutive disk blocks. Keeping recently read blocks in memory can reduce repeated storage accesses and support read-ahead.



Buffer Cache Organization

- The cache contains entries representing blocks currently held in memory.
- Each entry can maintain information such as:
  - Device / storage identifier
  - Block number
  - Valid or present status
  - Dirty/modified status
  - Reference or usage information
  - Locking/synchronization information
- Efficient lookup is important because every block request may perform a cache lookup.

Design issue

The OS must balance cache size, lookup overhead, memory pressure, and the benefit of keeping useful blocks in RAM.

Replacement Policies

When the cache is full, the OS must select an entry to evict.

LRU – Least Recently Used Evict a block that has not been used for the longest time.

Clock / Second Chance Approximate LRU using a reference bit and a circular scan.

Other policies Variants can consider frequency, recency, workload characteristics, or write behavior.

Important

No replacement policy is universally optimal. The appropriate policy depends on the access pattern and implementation cost.

Write Policies: Write-Through vs. Write-Back

Write-Through

- Update the cache and storage immediately.
- Storage remains relatively up to date.
- Simpler consistency behavior.
- Can generate more I/O operations.

Write-Back

- Modify the cached block first.
- Mark it **dirty**.
- Write it to storage later.
- Can reduce storage writes.
- Requires careful recovery and synchronization.

Example

If a process modifies the same block repeatedly, write-back can combine multiple modifications into fewer storage writes.

Read-Ahead and Delayed Write

Read-Ahead

- Predict that additional blocks will soon be requested.
- Load them into memory before the process explicitly asks for them.
- Particularly useful for sequential access.

Delayed Write

- Keep modified blocks in memory temporarily.
- Combine or reorder writes.
- Write data to storage later.
- Improves efficiency but introduces durability considerations.

Trade-off

Aggressive caching can improve performance but increases the amount of important state held in volatile memory.

Why Do We Need I/O Scheduling?

- Multiple processes can generate I/O requests simultaneously.
- A storage device may have many pending requests.
- The OS needs a policy to decide the service order.
- For magnetic disks, the order can significantly affect seek time and total response time.

Scheduling objectives

- Reduce average waiting/response time
- Increase I/O throughput
- Reduce mechanical movement for HDDs
- Provide reasonable fairness
- Meet deadlines or priority requirements when necessary

Disk Access Time Components

For a traditional magnetic disk:

$$T_{\text{access}} = T_{\text{seek}} + T_{\text{rotation}} + T_{\text{transfer}}$$

Seek time Time required to move the disk arm to the required track.

Rotational latency Time waiting for the required sector to rotate under the head.

Transfer time Time required to actually transfer the requested data.

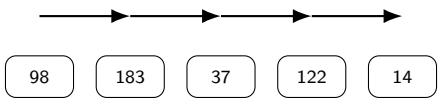
Scheduling insight

Disk scheduling primarily attempts to reduce **seek overhead**, although modern storage devices and SSDs change the relative importance of these components.

FCFS Disk Scheduling

First-Come, First-Served (FCFS)

- Requests are served in their arrival order.
- Simple and fair in the sense of preserving arrival order.
- Does not attempt to minimize head movement.



Limitation

A request sequence can cause unnecessary head movement and therefore high average access time.

SSTF Disk Scheduling

Shortest Seek Time First (SSTF)

- Select the pending request closest to the current head position.
- Usually reduces immediate seek distance.
- Can improve average response time compared with FCFS for many workloads.
- A far-away request may wait for a long time.

Example

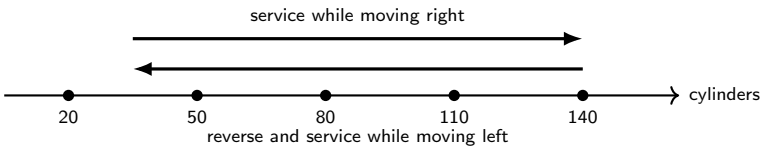
If the current head is at cylinder 50 and pending requests are 45, 52, 90, and 130, SSTF selects 52 first because it is closest to 50.

Concern

Repeated arrival of nearby requests can cause **starvation** of distant requests.

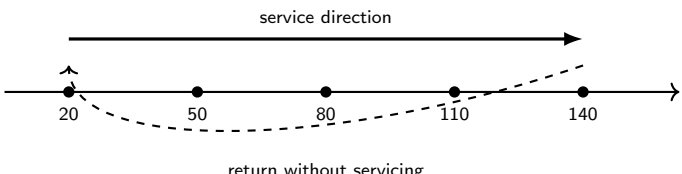
SCAN: The Elevator Algorithm

- The disk head moves in one direction and services requests along its path.
- When it reaches the end of the relevant range, it reverses direction.
- Similar to an elevator serving floors in both directions.
- Provides more predictable waiting behavior than SSTF.



C-SCAN: Circular SCAN

- The head services requests while moving in one direction only.
- After reaching the end, it returns to the beginning without servicing requests during the return.
- The process then continues in the same direction.
- Provides more uniform waiting time across the disk.



Disk Scheduling: Comparison

Algorithm	Main idea	Advantage	Concern
FCFS	Arrival order	Simple, predictable	High seek movement
SSTF	Closest request first	Often lower seek time	Possible starvation
SCAN	Sweep both directions	Good throughput/fairness balance	End-point behavior matters
C-SCAN	Sweep one direction	More uniform waiting	Extra return movement

Important

The best scheduling policy depends on workload, device technology, fairness requirements, and performance objectives.

LOOK and C-LOOK

LOOK

- Similar to SCAN.
- Reverses direction at the last pending request.
- Does not travel unnecessarily to the physical end of the disk.

C-LOOK

- Similar to C-SCAN.
- Services requests in one direction.
- Jumps from the last pending request to the first pending request.

Design improvement

LOOK and C-LOOK can reduce unnecessary movement when the request queue does not extend to the physical disk boundaries.

Worked Example: Choosing a Disk Request

Current head position: 50
Pending requests:

82, 170, 43, 140, 24, 16, 190

- **FCFS:** follows the listed order.
- **SSTF:** initially chooses 43 because it is closest to 50.
- **SCAN:** services requests while sweeping in a selected direction.
- **C-SCAN:** services requests in one direction and wraps around.

Analysis question

Which algorithm should be selected if the primary concern is predictable waiting time rather than only minimizing immediate seek distance?

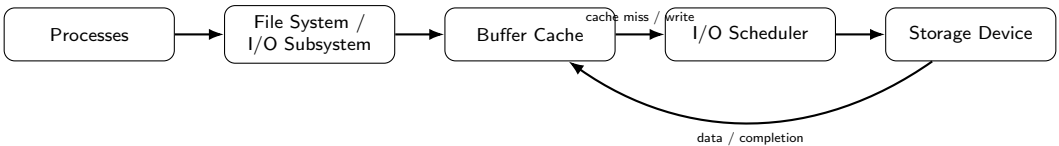
I/O Scheduling Beyond Hard Disks

- Traditional HDD scheduling is strongly influenced by mechanical seek and rotational latency.
- SSDs have no mechanical disk head, so seek-based optimization has much less relevance.
- Modern storage stacks may instead consider:
  - Queue depth
  - Parallelism
  - Request merging
  - Fairness
  - Read/write latency
  - Device-specific characteristics

Key point

An OS scheduling policy should reflect the characteristics of the underlying I/O device rather than blindly applying a policy designed for a different device technology.

Buffer Cache + I/O Scheduling: Integrated View



- Cache can eliminate some physical I/O requests completely.
- The scheduler orders requests that actually reach the device.
- Together, caching and scheduling improve overall I/O efficiency.

Key Trade-offs

- **High cache size** → potentially higher hit rate, but greater memory consumption.
- **Aggressive read-ahead** → fewer future misses, but potentially wasted I/O.
- **Write-back** → fewer storage writes, but greater durability/recovery complexity.
- **SSTF** → low local seek distance, but possible starvation.
- **SCAN/C-SCAN** → more predictable service, but potentially more movement.
- **Complex scheduling** → potentially better utilization, but higher implementation overhead.

OS design principle

Performance optimization is a trade-off among **latency, throughput, fairness, memory usage, device utilization, and reliability**.

Real-World Example: Opening a Large File

1. Application requests file data.
2. File-system code determines the required disk blocks.
3. OS checks the buffer/page cache.
4. Cached blocks are returned immediately on hits.
5. Missing blocks generate storage requests.
6. I/O scheduler orders pending requests.
7. Storage device transfers data, often using DMA.
8. Data is placed in memory and supplied to the application.

Big picture

Cache reduces how often storage must be accessed; **scheduling** determines how pending storage work is ordered; **DMA** can reduce CPU involvement in the transfer.

Quick Check / Discussion

1. What is the difference between a buffer and a cache?
2. Why does a high cache hit ratio improve I/O performance?
3. What is the purpose of a dirty bit in a buffer-cache entry?
4. Why can SSTF cause starvation?
5. How does SCAN differ from C-SCAN?
6. Why are traditional disk-scheduling algorithms less dependent on seek time for SSDs?

Think

Suppose an application performs sequential reads of a very large file. Which cache and I/O strategies could exploit this access pattern, and what risks might aggressive read-ahead introduce?

Key Takeaways

- **Buffer cache** keeps useful storage blocks in RAM to reduce expensive device accesses.
- Cache performance depends strongly on **locality** and replacement policy.
- **Write-through** and **write-back** provide different consistency and performance trade-offs.
- **I/O scheduling** determines the order in which pending device requests are serviced.
- FCFS is simple; SSTF can reduce seek distance; SCAN and C-SCAN improve service regularity.
- Modern storage requires scheduling policies appropriate to the device technology.
- Efficient I/O is achieved by combining **caching, buffering, scheduling, and efficient data transfer**.

Cache fewer accesses. Schedule the remaining work intelligently.